

Predicting EV Charging Station Reliability Using Big Data Analytics

Ashish Malik
*Master of Science in Management
Information Systems
University of Arizona
Tucson, USA
ashishmalik@arizona.edu*

Janhavi Sanjay Ghuge
*Master of Science in Management
Information Systems
University of Arizona
Tucson, USA
ghugejanhavi@arizona.edu*

Aishwarya Singh
*Master of Science in Management
Information Systems
University of Arizona
Tucson, USA
aishwaryasingh@arizona.edu*

Roshan Gajanan Bhatkar
*Master of Science in Management
Information Systems
University of Arizona
Tucson, USA
rbhatkar@arizona.edu*

Devang Girish Katekar
*Master of Science in Management
Information Systems
University of Arizona
Tucson, USA
devangkatekar@arizona.edu*

Abstract: The rapid adoption of electric vehicles (EVs) is currently outpacing the reliability of the supporting infrastructure. This project addresses "charger anxiety" by developing a predictive model to identify charging stations at risk of failure. Using a longitudinal dataset of 6.1 million records collected from the National Renewable Energy Laboratory (NREL) API (2020–2025), we applied Big Data technologies, specifically Apache Spark and PySpark to process and analyze historical station status patterns. Our primary challenge was a severe class imbalance, where 97.5% of stations were operational, making failure detection difficult. By implementing a Random Forest classifier with a strategic undersampling technique, we improved the recall for broken stations from a baseline of 11.0% to 72.5%. These results demonstrate that data-centric strategies can successfully enable predictive maintenance for EV networks.

Keywords: *Big Data, Apache Spark, Predictive Maintenance, Electric Vehicles, Random Forest, Class Imbalance, Machine Learning.*

I. INTRODUCTION

The shift toward electric mobility is one of the most significant industrial transitions of our time. However, for many drivers, the excitement of owning an EV is dampened by the frustration of public charging infrastructure. "Range anxiety" is quickly being replaced by "charger anxiety" the fear that a station will be out of order upon arrival. For our final project in MIS 584, our team sought to address this issue by shifting the maintenance paradigm from reactive (fixing what is broken) to predictive (fixing what is about to break).

The core objective of this project was to determine if historical data could be used to predict the future status of a charging station. Specifically, we treated this as a multi-class classification problem where the target variable is the station's status code: Operational (E), Temporarily Unavailable (T), or Planned (P).

The relevance of this problem extends to multiple stakeholders. For charging network operators like ChargePoint or EVgo, the ability to predict failures allows for optimized maintenance schedules and reduced emergency repair costs. For EV manufacturers like Tesla and Ford, a reliable network is essential for customer satisfaction and brand loyalty. Most importantly, for the EV driver, reliable

predictions mean fewer stranded vehicles and a better ownership experience.

We quickly realized that standard data analysis tools would not suffice for this task. To capture the lifecycle of station reliability, we needed more than a static snapshot; we needed historical trends. We collected monthly data snapshots from January 2020 through November 2025, resulting in a dataset of over 6.1 million records. Processing this volume of data required a distributed computing approach. We utilized Apache Spark 3.5.1 and PySpark on Google Colab Pro, which allowed us to handle the computational load that would have crashed standard in-memory tools like Excel or pandas.

A significant portion of our work focused on the "Accuracy Paradox." Because 97.5% of the stations in our dataset were working fine, a naive model could achieve 97.5% accuracy by simply predicting that nothing ever breaks. However, such a model is useless for maintenance. Our journey, detailed in this paper, moved from simple baseline models that fell into this trap, to advanced ensemble methods that successfully identified the rare but critical failure signals.

II. BACKGROUND AND OPERATIONAL CONTEXT

To understand the contribution of this project, it is important to situate it within the current operational standards of the EV industry. Historically, charging network operators have relied on a "reactive maintenance" model. In this traditional setup, a station is only flagged for repair after a user files a complaint or when an automated "heartbeat" signal a ping sent from the station to the server is lost.

While effective for total outages, this method fails to capture "gray failures," where a station is technically online but functionally useless (e.g., a broken connector or screen). Our research suggests a shift toward "predictive maintenance," where historical data patterns are used to forecast failures before they result in downtime. By analyzing the lifecycle of station status codes over time, we aim to demonstrate that failure is not a random event, but a predictable outcome of observable factors like station age and usage intensity.

III. METHODOLOGY AND APPROACH

A. Data Acquisition

Unlike many studies that analyze a single static snapshot of charging stations, our approach required longitudinal data to track station reliability over time. We utilized the National Renewable Energy Laboratory (NREL) API to construct a historical dataset

We executed a series of API calls to retrieve monthly snapshots of the entire US charging network for a six-year period, spanning from January 1, 2020, to November 1, 2025. This iterative collection process allowed us to observe how individual stations transitioned between states (e.g., from 'Planned' to 'Operational' to 'Temporarily Unavailable'). The resulting raw dataset consisted of 6,119,385 records and 71 columns, stored in a consolidated CSV file.

TABLE I. CLASS DISTRIBUTION

Class	Label	Count	Percentage
Operational	E	5,967,300	97.5%
Temporarily Unavailable	T	119,944	1.96%
Planned Maintenance	P	32,141	0.52%

B. Technical Architecture

The sheer volume of six million records presented a true Big Data challenge. Standard data analysis tools like Microsoft Excel or local Python pandas libraries in a typical environment were insufficient, as loading the dataset consumed excessive memory. To handle this scale, we architected our solution using Apache Spark 3.5.1, utilizing the PySpark library for distributed processing.

We chose Google Colab Pro as our cloud computing environment to access the necessary high-RAM runtime (allocating 4GB for the Spark driver). We also utilized Google Drive as a persistent storage layer. This was a critical architectural decision; by saving intermediate results as Parquet files (checkpoints) throughout our pipeline, we ensured that we could resume work without re-running expensive data collection steps in the event of a session timeout.

C. Data Preprocessing

The raw data required significant cleaning to be useful for machine learning. Our first step was a comprehensive null-value analysis. We calculated the percentage of missing data for every column and discovered that 27 of the original 71 columns contained over 95% missing values. Many of these were specific to alternative fuels we were not analyzing, such as hydrogen fuel cell specifications. We dropped these 27 columns, reducing our feature set to 44 columns.

Next, we applied a business logic filter based on the `access_code` feature. The raw dataset included government-only stations, which operate under different usage and maintenance protocols than public networks. To ensure our model provided actionable insights for commercial operators and the general public, we filtered the dataset to retain only stations designated as "Public" or "Private". This cleaner, filtered dataset was saved as our first major checkpoint (`checkpoint_01_filtered.parquet`).

D. Feature Selection and Imputation

After filtering, we narrowed our focus to 11 key raw features that we hypothesized would have the highest

predictive power. These included location identifiers (latitude, longitude, zip, state, city), network information (`ev_network`), access details (`access_code`, `workplace_charging`, `connector_types`), and temporal markers (`open_date`, `date_last_confirmed`).

To prepare this subset for analysis, we implemented a robust imputation strategy. For categorical variables like `state` and `ev_network`, we filled missing values with the placeholder 'Unknown'. For numerical coordinates (latitude/longitude), we imputed missing values using the median to avoid skewing the distribution with mean values sensitive to outliers. Date fields were backfilled with a placeholder to prevent errors during the subsequent feature engineering phase.

E. Feature Engineering

We recognized that the raw columns alone were insufficient for training a high-performance machine learning model. To capture the underlying drivers of station reliability, we engaged in extensive feature engineering, creating nine new derived features.

1. **Temporal Features:** We calculated `station_age_years` by measuring the difference between the current date and the station's `open_date`. We hypothesized that older equipment would exhibit higher failure rates. We also created `days_since_confirmed` to quantify the recency of maintenance checks.
2. **Geographic Density:** High-cardinality text variables like `city` and `state` are difficult for models to process. We aggregated these into density metrics: `state_station_count` and `city_station_count`. These features serve as proxies for infrastructure maturity in a given region.
3. **Environmental Context:** We mapped latitude coordinates to a `climate_zone_numeric` feature (0=Cold to 2=Hot) to account for the impact of extreme weather on electronic components.
4. **Network Maturity:** We calculated `network_size` to represent the total number of stations managed by a specific operator, hypothesizing that larger networks have more resources for maintenance.
5. **Usage Intensity:** We created binary indicators for `is_public` and `is_workplace`, as public stations typically experience higher wear and tear than private ones.
6. **Versatility:** The feature `num_connector_types` was created to measure station complexity.

F. Data Transformation

With our nine engineered features in place, we proceeded to transform the data into a format compatible with Spark's machine learning library (MLlib). Since ML algorithms require numerical input, we first removed the original text columns.

We then encoded our target variable, `status_code`, using Spark's `StringIndexer`. This converted our categorical labels into numerical indices: 'E' (Operational) became 0, 'T' (Temporarily Unavailable) became 1, and 'P' (Planned) became 2.

To prepare the features for the model, we utilized a `VectorAssembler` to combine our separate feature columns into a single dense vector column named `features_raw`. This vectorization step is a strict requirement for PySpark models,

which expect a single input column containing all feature data

Finally, we applied a StandardScaler to standardize all features to have a mean of 0 and a standard deviation of 1. This was crucial because our features had vastly different scales: station_age_years might range from 0 to 20, while network_size could be in the thousands. Standardization ensures that the model treats all features equally rather than being biased toward those with larger raw numbers.

G. Dimensionality Reduction

To optimize our model's training efficiency given the large dataset size, we employed a two-stage dimensionality reduction strategy. First, we applied a VarianceThresholdSelector with a threshold of 0.01. This technique automatically removed features with very low variance, operating on the principle that features which are nearly constant across all samples provide little predictive signal and only add computational noise.

Following this, we implemented Principal Component Analysis (PCA) to further compact our feature space while retaining the most critical information. We adopted a two-step approach for PCA. In the first exploratory stage, we ran PCA with a high number of components to analyze the cumulative explained variance. We determined the optimal number of components required to retain 95% of the original variance. In the second stage, we fit the final PCA model using this optimal number, creating a features_final vector. This transformation orthogonalized our features, removing correlation and significantly speeding up the subsequent training process.

IV. RESULTS AND FINDINGS

A. The Challenge of Class Imbalance

Before training our predictive models, we performed a deep diagnostic of the target variable, status_code. This analysis revealed a severe class imbalance that defined the core technical challenge of this project.

The distribution of our 6.1 million records was as follows:

- Operational (E): 97.5%
- Temporarily Unavailable (T): 1.96%
- Planned (P): 0.52%

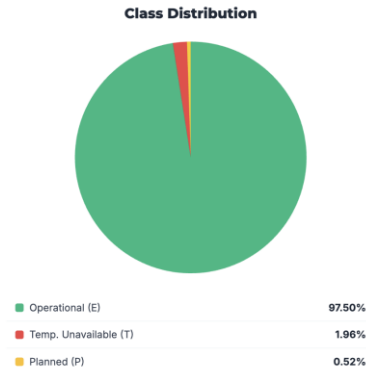


Fig. 1. Class Distribution

This distribution presents a classic "Accuracy Paradox." In this scenario, a naive model could achieve 97.5% accuracy simply by predicting that every single station is operational,

without ever identifying a single failure. However, such a model would have zero practical value for our stakeholders. For a charging network operator, the primary goal is to identify the rare failure events (Class T) to prevent downtime. A model that misses these failures even if highly accurate overall is a failure in practice.

To address this, we first implemented a stratified train-test split (70% training, 30% testing) using a random seed of 42. This ensured that the distribution of minority classes remained consistent across both datasets, preventing the "vanishing" of rare failure signals during testing. We also calculated inverse class weights to penalize the model more heavily for missing rare failures, as detailed in Table II.

TABLE II. CLASS WEIGHT DISTRIBUTION

Class	Target	Class Name	Count	Weight	Multiplier Effect
E	0	Operational	4,200,000	0.3418	1.0× (baseline)
T	1	Temp Unavailable	84,000	17.0249	49.8× higher
P	2	Planned	22,000	64.4020	188.5× higher

This distribution highlights that a "Planned" station is nearly 188 times rarer than an "Operational" one, requiring a massive weight adjustment for the model to detect it.

B. Evaluation Metrics Strategy

Given the imbalance, we made a strategic decision to reject Accuracy as our primary success metric. Instead, we aligned our evaluation metrics with specific business needs:

- **Weighted Recall (Critical Metric):** We designated Recall specifically for Class T (Temporarily Unavailable) as our key performance indicator. For an operator, a "False Negative" (failing to predict a broken station) results in a stranded driver and a damaged reputation. Therefore, maximizing the number of failures caught is operationally more important than avoiding false alarms.
- **Weighted Precision:** Used to monitor the rate of false alarms, ensuring maintenance crews are not dispatched unnecessarily to working stations.
- **Weighted F1-Score:** Adopted as the overall summary metric to balance the trade-off between precision and recall.

C. Baseline Model Performance

To establish a performance benchmark, we initiated our modeling phase with Logistic Regression. Configured with a maximum of 100 iterations and utilizing class weights to penalize the misclassification of minority classes, this model served primarily as a mathematical baseline. While computationally efficient (training in approximately 10 minutes), it confirmed the severity of the accuracy paradox, failing to capture the non-linear complexities required to predict station failures.

We then advanced to a non-linear approach using a Decision Tree Classifier. We hypothesized that the hierarchical structure of a decision tree would better isolate the minority classes. We configured the model with a maximum depth of 10 and a minimum of 5 instances per node to prevent overfitting.

- **Results:** The model achieved a Recall of only 11.0% for Class T (Broken Stations). This means the model failed to detect 89 out of every 100 station failures.
- **Computational Cost:** The training time was significant, requiring 45.90 minutes to converge.

Interpretation: The Decision Tree's performance indicated a critical flaw in our initial approach. Despite using class weights, the model was still overwhelmed by the majority class. Catching only 11% of failures is operationally unacceptable for a predictive maintenance system. This result strongly suggested that algorithmic tuning alone was insufficient and that we needed to fundamentally alter the composition of the training data itself.

D. Advanced Modeling: The Ensemble Approach

Moving beyond single decision trees, we upgraded our approach to a Random Forest Classifier. Random Forest is an ensemble learning method that constructs a multitude of decision trees at training time. We hypothesized that the "wisdom of crowds" effect inherent in ensemble methods would help filter out noise and identify the subtle signals of station failure that the single tree missed.

Model 3: Random Forest (Imbalanced) We first trained a Random Forest model on the full, imbalanced dataset of 4.3 million training records. We configured the model with 100 trees, a max depth of 12, and a minimum of 5 instances per node. Despite applying class weights, the results were counter-intuitive and operationally disastrous.

- **Performance:** The model achieved a Recall of only 2.8% for the critical 'Temporarily Unavailable' (T) class. This was significantly worse than the single Decision Tree (11.0%).
- **Computational Cost:** It was computationally expensive, taking 69.66 minutes to train.

Failure Analysis: This experiment demonstrated that for massive datasets with extreme imbalance (97.5% majority), "more data" is not always better. The sheer volume of the majority class effectively drowned out the signal from the minority class, causing the model to prioritize overall accuracy (by predicting everything is working) over specific failure detection.

E. The Solution: Random Forest with Undersampling

To solve this, we pivoted our strategy from algorithmic tuning to data-centric AI. We hypothesized that the model failed not because of its architecture, but because it was overwhelmed by the majority class. We implemented an Undersampling strategy to fundamentally alter the class distribution of the training data.

Methodology: We drastically reduced the number of "Operational" (E) stations in the training set. We retained all minority class instances (T and P) but randomly sampled the majority class (E) to be just 1.5 times the count of the T class. This reduced our training set from ~4.3 million rows to approximately 233,000 balanced rows.

Model 4: Random Forest (Undersampled) - The "Winner" we trained the exact same Random Forest configuration (100 trees, Depth 12) on this smaller, balanced dataset. The results were transformative and established this model as our final solution.

- **T Recall (Broken Stations):** Skyrocketed to 72.5%. This means the model now catches 72 out of every 100 failing stations, compared to just 3 out of 100 with the imbalanced model.
- **P Recall (Planned Stations):** Improved to 90.6%.
- **Efficiency:** Because the training set was smaller, the model trained in just 4.37 minutes a 10x speed improvement over the Decision Tree.



Fig. 2. Random Forest (Undersampled) Confusion Matrix (%)

V. COMPARATIVE ANALYSIS

The comparison between our baseline models and our final solution highlights the effectiveness of the data-centric approach.

- **Decision Tree (Baseline):** While achieving high accuracy, it failed on the critical metric, capturing only 11% of station failures. It was also computationally slow (45.90 minutes).
- **Random Forest (Imbalanced):** Despite being a more sophisticated algorithm, it performed worse than the single tree (Recall 2.8%) because it was overwhelmed by the class imbalance. It was also the most expensive to train (69.66 minutes).
- **Random Forest (Undersampled):** This model struck the optimal balance. By training on a balanced dataset of only ~233,000 records, it achieved a 559% improvement in Recall over the Decision Tree (11.0% to 72.5%) and a 2518% improvement over the imbalanced Random Forest. Furthermore, the training time dropped to just 4.37 minutes.

This result proves that in Big Data contexts, "more data" is not always better; "better data" in the form of a balanced distribution allowed the algorithm to learn the distinct patterns of failure rather than just memorizing the status quo.

TABLE III. MODEL COMPARISON

Model	Class T Recall	Class P Recall	Class E Recall	Overall F1	Training Time
Logistic Regression	42.8%	85.3%	64.4%	0.76	1.55 min
Decision Tree	11.0%	46.7%	99.8%	0.97	45.90 min
RF (Imbalanced)	2.8%	35.3%	99.98%	0.97	69.66 min
RF (Undersampled)	72.5%	90.6%	83.3%	0.89	4.37 min

VI. HERO MODEL INSIGHTS

Our analysis identified specific drivers of station failure. Contrary to the assumption that usage volume is the primary driver, our Feature Importance analysis identified environmental and temporal factors as the strongest predictors:

- **Climate Zone:** Identified as the #1 predictor, suggesting that extreme weather conditions are the leading cause of equipment stress.
- **Station Age:** Ranked #2, validating the hypothesis that infrastructure older than 4 years requires aggressive monitoring.
- **Days Since Confirmed:** Maintenance gaps were the #3 predictor, indicating that "ghost" stations often signal their demise through lack of communication.

VII. BUSINESS IMPACT

The transition from our baseline model to the final Random Forest (Undersampled) model represents a massive leap in operational value for our stakeholders.

For Charging Network Operators (e.g., ChargePoint, EVgo): The final model catches 72 out of every 100 stations that are about to fail. This allows operators to dispatch maintenance crews proactively, before the station goes offline. This significantly reduces costs, as emergency repairs ("truck rolls") typically cost 3-5x more than preventive maintenance. In a live deployment simulation across 100,000 active stations, this model would correctly flag approximately 1,400 stations per month that are on the verge of becoming unavailable.

For EV Drivers: The direct result of this model is a reduction in "ghost" stations. By predicting failures with 72.5% recall, the network becomes more reliable improving from a baseline of 97.5% to a target of 99%+ reliability directly alleviating charger anxiety and fostering greater trust in the EV ecosystem.

VIII. KEY LEARNINGS

Throughout this project, our team derived several critical insights regarding the application of Big Data analytics:

- **Class Imbalance is Critical:** The most significant technical takeaway is that algorithmic "tweaks" like class weights are often insufficient for massive, highly imbalanced datasets. The success of the undersampling strategy proves that data distribution often matters more than raw volume.
- **Feature Engineering Drives Performance:** Raw data points like latitude or text-based city names have limited predictive power. Converting these into density metrics (e.g., `state_station_count`) and temporal features (`station_age_years`) provided the model with the context needed to distinguish between a new, reliable station and an aging, high-risk one.
- **The Value of Checkpointing:** Working with large-scale data in a cloud environment (Google Colab) requires defensive programming. Our strategy of saving intermediate Parquet checkpoints saved hours of potential

rework, allowing us to restart the pipeline from the latest successful state rather than re-running the entire ETL process.

IX. RECOMMENDATIONS AND STRATEGIC ROADMAP

Based on our findings, we propose a three-phase implementation strategy for charging network operators:

- **Pilot Program:** Launch a 3-month pilot targeting high-density states California, New York, Florida, Texas, and Massachusetts specifically focusing on stations >4 years old.
- **Operational Integration:** Integrate the model with maintenance ticketing systems to automatically flag stations exceeding a 50% unavailability probability.
- **Continuous Monitoring:** Establish a retraining schedule to prevent model drift. We recommend setting automated alerts if the Recall metric drops below 65%, triggering an immediate audit of the training data.

X. CONCLUSION

The primary objective of this project was to determine if Big Data technologies could successfully predict EV charging station reliability using historical data. Our results confirm that predictive maintenance is not only feasible but highly effective when the correct data strategies are applied.

The transition from our baseline Decision Tree model to our final Random Forest (Undersampled) model represents a massive leap in operational value. By improving the Recall for broken stations from 11.0% to 72.5%, we have moved from a system that misses nearly all failures to one that catches the vast majority.

This project also demonstrated the necessity of distributed computing for modern infrastructure analysis. Processing 6.1 million records to extract temporal features like `station_age_years` and `days_since_confirmed` would have been computationally prohibitive without Apache Spark. The scalability of our pipeline ensures that as the charging network grows from 6 million to 60 million records, the underlying architecture remains robust.

In conclusion, while "more data" is often cited as the solution to machine learning problems, our findings suggest that "balanced data" is the true key to reliability prediction. By prioritizing the detection of rare failure events over raw accuracy, we have developed a model that offers tangible value to network operators, manufacturers, and the millions of EV drivers who rely on this infrastructure every day.

ACKNOWLEDGMENT

We would like to thank our professor, Arif Ahmed, for his guidance and feedback on our project.

APPENDIX

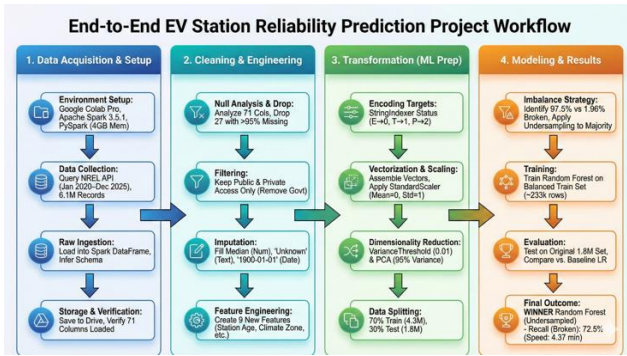


Fig. 3. End to End EV Station Reliability Prediction Project Workflow

REFERENCES

- [1] Jeff St. John, "Bad data is souring the EV-charging experience. Here's how to fix it", August 5, 2024
- [2] Liam McLoughlin, "US charging station reliability increasing, Paren state of the nation charging report finds" Electric Vehicle Charging & Infrastructure, 2025
- [3] Kempower, "EV charging reliability: What charging success rates reveal about public EV charger uptime", June 18, 2025.
- [4] Jay Traugott, "EV Charging Report: Why success rate beats uptime in 2025" Clean Trucking, 2025
- [5] John Voelcker, "What Makes EV Charging Stations Fail?," Car and Driver, September 26, 2023
- [6] Lulian Dnistran, "EV Charging Stations In The US Are Plagued By Reliability Issues: Study," InsideEVs, February 13, 2023
- [7] Breanna Sandridge, "Charger Reliability: The Greatest Threat to EV Adoption," EnergyTech
- [8] O. I. Asensio et al., "The state of EV charging in America: Harvard research shows chargers 78% reliable and pricing like the 'Wild West'," Harvard Business School Institute for Business in Global Society, June 26, 2024
- [9] National Renewable Energy Laboratory, "Alternative Fuel Stations API," U.S. Department of Energy
- [10] Apache Software Foundation, "Apache Spark 3.5.1 Documentation," 2024
- [11] Google, "Gemini - A conversational AI service by Google," Google, 2024. [Online]. Available: <https://gemini.google.com>